RICHTLINIE

Projektstruktur

Verzeichnisse, Skripte und erzeugte Dateien des Gesamtprojekts: welcher Pfad wofür zuständig ist

Dieses PDF gibt die Quelle im Wortlaut wieder; Listen, Tabellen und Codeblöcke entsprechen der Quelle, weich umbrochene Zeilen sind zu Absätzen zusammengeführt.

Quelle	C8205-Projektstruktur.md	0.95-029
Stand	25.10.2026	
Umfang	17 Seiten, A4 hoch	
Erzeugt mit	erzeuge_projektstruktur_pdf.py	

> Maßgeblich bleibt die Quelldatei. Dieses PDF wird nicht automatisch nachgeführt: Es ist aktuell, solange die Version oben mit der Versionszeile der Quelldatei übereinstimmt.

Inhaltsverzeichnis

1. Oberste Ebene 3
1a. Die Skripte in testscripte/ 12
2. Konventionen für Tests, Protokolle und virtuelle Geräte 14
3. Teilprojekt Assembler 14
3a. Teilprojekt BIS 16
3b. Experimenteller Prototyp C8205Z 16
4. Offene Punkte 17

C 8205 – Projektstruktur

Quelle: C8205-Projektstruktur.md · im Wortlaut der Quelle

Version: 0.95-029

Dieses Dokument erläutert die Verzeichnisstruktur des Gesamtprojekts C 8205: welches Verzeichnis welchen Zweck hat und in welchem Dokument die Einzelheiten stehen. Es wiederholt keine Details aus den Teildokumenten, sondern verweist nur auf sie (wie "C8205-Interpreter-Überblick_und_Gesamtaufbau.md"). Erzeugte Dateien werden nur nach Endung beschrieben, nicht einzeln aufgezählt.

Das Projekt hat drei Teilprojekte: den **Interpreter** (Python, Wurzel des Projektverzeichnisses), den **Assembler** (C, Unterverzeichnis `Assembler/`) und das **Bausteininterpretationssystem BIS** (Programmsystem für den C 8205; derzeit nur Dokumentation, s. Abschnitt 3a).

Dazu kommt der **experimentelle Prototyp C8205Z** (die vier Zusatztrommeln des C 8205Z, Unterverzeichnis `C8205Z/`, Abschnitt 3b). Er ist kein Teilprojekt des Simulators, sondern ein experimenteller Prototyp; im Projektverzeichnis ist `C8205Z/` der einzige Pfad dafür. Die Anbindung an Interpreter und GRAFIK (`c8205_zt.py`, `grafik_zt.py` und die Schalterzeilen in den Dokumenten) liegt im Simulator selbst und wirkt nur mit dem Schalter `--C8205Z`.

1. Oberste Ebene

Pfad	Zweck	Maßgebliches Dokument
`c8205_interp.py`	Python-Interpreter (Knotenautomat)	`C8205-Interpreter-Python_Interpreter.md`
`c8205_zt.py`	Anbindung des **experimentellen Prototyps C8205Z** an den Python-Interpreter (Zusatztrommeln ZT1 bis ZT4, Zeit 0, nur Modus TEST); wird nur mit dem Schalter `--C8205Z` geladen	`C8205-Interpreter-Python_Interpreter.md` (Schalterzeile), `C8205Z/zt/Zusatztrommel_Zeitmodell.md`
`grafik-interpreter-module/grafik_zt.py`	Anbindung des **experimentellen Prototyps C8205Z** an GRAFIK (Schalter `--C8205Z`): benutzt das C-Modul `C8205Z/zt_modul` über das Erweiterungsmodul `c8205_ztmodul` (Zeitmodell 81 bis 86 ms je Spur)	`C8205-Interpreter-GRAFIK_Beschreibung.md` §7.10
`C8205Z/`	**Experimenteller Prototyp C8205Z** (Zusatztrommeln), einziger Zusatzpfad des Projekts; Inhalt: Abschnitt 3b	`C8205Z/zt/Zusatztrommel_Zeitmodell.md`
`Trommel/ZT/`	Dateien `zt1.txt` bis `zt4.txt` der Zusatztrommeln (Format wie `Trommel/trommel.txt`); entstehen erst beim Lauf mit `--C8205Z` (fehlende Dateien werden leer angelegt)	`C8205Z/zt/Zusatztrommel_Zeitmodell.md`
`Hilfsprogramme/`	Ordner für die Hilfsprogramme des Interpreter-Teilprojekts: `c8205_lochstreifen_anzeige.py`, `c8205_lochstreifen_eingabe.py`, und `c8205_lochstreifen_umwandlung.py` (die drei folgenden Zeilen). Sie liegen nicht mehr in der Wurzel; `c8205_interp.py` und die Wrapper `c8205_interpreter.sh/.bat` bleiben dort	`C8205-Interpreter-Lochstreifen-Hilfsprogramme.md`
`Hilfsprogramme/c8205_lochstreifen_anzeige.py`	Hilfsprogramm (Interpreter-Teilprojekt): liest einen virtuellen Lochstreifen (Datei oder Verzeichnis, Format wie `--leser1`/`--leser2`) und schreibt ihn als Klartext in einem scrollbaren Fenster aus (Schalter `--LS-LESER`). `Um. KB`/`Um. GB` wirken nur, `Irrung Zeichen`/`Irrung Satz` löschen im Text. Eigene Programmversion (`--version`)	`C8205-Interpreter-Lochstreifen-Hilfsprogramme.md`

C 8205 – Projektstruktur

Quelle: C8205-Projektstruktur.md · im Wortlaut der Quelle

Pfad	Zweck	Maßgebliches Dokument
`Hilfsprogramme/c8205_lochstreifen_eingabe.py`	Hilfsprogramm (Interpreter-Teilprojekt), Gegenstück zur Anzeige: erzeugt einen virtuellen Lochstreifen aus Tastatureingaben in einem Fenster (Schalter `--LS-STANZER`); Ergebnis geeignet für `--leser1`/`--leser2`. Eigene Programmversion (`--version`)	`C8205-Interpreter-Lochstreifen-Hilfsprogramme.md`
`Hilfsprogramme/c8205_lochstreifen_umwandlung.py`	Hilfsprogramm (Interpreter-Teilprojekt): wandelt eine Textdatei in einen binär kodierten virtuellen R300-Lochstreifen um (`--IN-BIN`, Vorgabe) oder umgekehrt (`--IN-TXT`); Aufruf `python3 c8205_lochstreifen_umwandlung.py --LS-STANZER AUSGABE --LS-LESER EINGABE [--IN-BIN | --IN-TXT]`	Kopfkommentar der Datei
`c8205_grafik.py`, `c8205_grafik.sh`, `c8205_grafik.bat`	Treiber des Modus GRAFIK (Pult-Fenster, SW-Fenster, Töne); Schalter u. a. `--STUMM`, `--TON=pppp` (Pult, SW, LL, LS; Vorgabe 5555), `--leser1`, `--leser2`, `--stanzer1`, `--HWCHECK`; Rückkehrcode 16 = Start nicht möglich (z. B. kein Abspielprogramm)	`C8205-Interpreter-GRAFIK_Beschreibung.md` (Abschnitte 1, 2, 6)
`grafik-interpreter-module/grafik_*.py`	Module des Modus GRAFIK (liegen seit G5-002 in diesem Unterordner; `c8205_grafik.py` und die Tests tragen ihn in `sys.path` ein). Kern und Geräte: `grafik_trommel`, `grafik_geraete` (Leser, Stanzer), `grafik_k0`. Fenster: `grafik_pult`, `grafik_pult_fenster`, `grafik_sw`, `grafik_sw_fenster`, `grafik_hwcheck`, `grafik_fensterlage`, `grafik_tastatur`, `grafik_haltzeit`, `grafik_debuglog`. **Töne (Stufe 4):** `grafik_tonmisch` (Mischer, Python), `grafik_tonmisch_c` (Mischer über das C-Modul), `grafik_schreibwerk` (Schreibwerkzeit 80 ms ±10 ms, Zeichenklang 100 ms, TAB 400 ms, WRZL 360 ms), `grafik_geraeteton` (Leser, Stanzer), `grafik_rechnerton` (Hochlauf, Grundgeräusch, Auslauf); `grafik_ton` bleibt für Windows (Stufe 2)	`C8205-Interpreter-GRAFIK_Beschreibung.md` (Abschnitte 5, 6, 11)
`c-module/`	C-Module: `trommel`, `bahnsperre`, `unterbrechung` und `tonmisch` (Mischer in Blöcken zu 10 ms mit eigenem C-Thread, statische Bibliothek, fest im Erweiterungsmodul), Testrahmen `testframe_*.c`, `Makefile`; `c8205_api.h` (Exportmakro der gemeinsamen Bibliothek), `plattform.h` (dünne Schicht für Linux und Windows, nur von `trommel.c` eingebunden; der Windows-Zweig ist nicht übersetzt und nicht getestet) und `build_msvc.bat` (Bau von `trommel.dll` mit Visual Studio, nicht getestet); `python/` mit dem Erweiterungsmodul `c8205_trommel` (`setup.py`, `c8205_trommel_modul.c`)	`C8205-Interpreter-GRAFIK_Beschreibung.md` §7

C 8205 – Projektstruktur

Quelle: C8205-Projektstruktur.md · im Wortlaut der Quelle

Pfad	Zweck	Maßgebliches Dokument
`ton/`	Klänge und Testrahmen: `sw_toene/` (24 Zeichenklänge, `tab_400.wav`, `wrzl.wav`, `wr.wav` (WR, 120 ms, aus den ersten 120 ms von `wrzl.wav`, erzeugt mit `erzeuge_wr_klang.py`), Leserklänge `ls_*.wav`; `tab_kurz.wav`/`tab_lang.wav` nur Quelle), `toene_rechner_stanzer/` (Stanzerklänge `ls_motor.wav` und `ls_dauerstanzen.wav`; die dortigen Kopien von Netz ein/aus und Grundgeräusch liest kein Programm), die drei Klänge des Rechners `netz_ein_fade_in.wav`, `netz_aus_fade_out.wav` und `rechner_grundgeraeusch.wav` liegen im Ordner `ton/` selbst, `sw_ton.py` (selbständiger Testrahmen: Schreibwerkzeit 80 ms ±10 ms, Zeichenklang 100 ms), `SW_Ton_Beschreibung.md`	`ton/SW_Ton_Beschreibung.md`
`pult/`	Bilder und Manifest des Bedienpult-Fensters: `pult_manifest.json`, `bild/` (`c8205_bedienpult.png`, das Bild des Pults) und `grafiken/` (Lampen, Dioden und Schlüsselstellungen als PNG)	`C8205-Interpreter-GRAFIK_Beschreibung.md` §3, §4
`testscripte/` (GRAFIK-Prüfungen)	Prüfungen des Modus GRAFIK: `grafik_alle_tests.sh` (alle Blöcke, `--schnell` ohne den Gleichwertigkeitstest), `grafik_stufe4_tests.py` (Töne), `grafik_sw_ende_zu_ende.py`, `grafik_sw_abstand_tests.py`, `grafik_tastatur_tests.py` u. a. Dazu `grafik_sektor_pruefung.py` (prüft, ob die Befehlsschleife schnell genug ist, damit zeitoptimal programmierter Code seinen Sektor findet; Abstandskarte aus Sprungketten), die Probe `grafik_trommel_zaehler_probe.py` (Zeitmodelle und Zähler der Trommelstatistik auf einer Kopie der Trommel) und das Messwerkzeug `k0_schatten_haken.py` (zu Interpreter 0.03-016, nur auf eine Kopie anzuwenden) und die Prüfung des Eingabeprogramms `r300_eingabe`: `r300_eingabe_erzeugen.py` (erzeugt Quelltext, Testauftrag, K0-Eingabedatei und die Klickspur `Tests/r300_eingabe_klickspur.txt`, die derzeit nicht im Projekt liegt), `r300_eingabe_trockenlauf.py` (ohne Tkinter), `r300_eingabe_grafik.sh`/`.bat` (GRAFIK-Lauf mit `--HWCHECK` und `--WIEDERGABE`), `r300_eingabe_lauf_und_auswertung.sh`/`.bat` (Lauf und Auswertung in einem Schritt) und `r300_eingabe_log_auswerten.py` (wertet das Debug-Log aus)	`C8205-Interpreter-GRAFIK_Pruefverfahren.md` §3

C 8205 – Projektstruktur

Quelle: C8205-Projektstruktur.md · im Wortlaut der Quelle

Pfad	Zweck	Maßgebliches Dokument
c8205_zip_packen.sh	Packt das Gesamt-Zip C8205 Simulator V.M.P.zip (Version aus der Release-Tabelle in C8205-Versionierung.md, Abschnitt 11, oder als Parameter). Vorher entstehen Test-Protokolle.zip und Regressionstest.zip; beide Ordner selbst kommen nicht ins Gesamt-Zip, die Sub-Zips schon. Mit --claude entsteht stattdessen die schlanke Variante C8205 Simulator V.M.P Claude.zip (ohne PDFs, die Testtabelle Testausfuehrungen/test_tabelle*, pult/, ton/, virtuelle_Geraete/, README.en.md, Test-Protokolle.zip, __pycache__ und die erzeugten .log .mqo .obj .sym unter Assembler/)	C8205-Versionierung.md, Abschnitt 12
lieferung_packen.sh	Packt eine Lieferung (Checkpoint) als Zip, jede Datei mit ihrem Projektpfad: --seit <Original.zip> nimmt alles Neue und Geänderte (ohne die erzeugten Dateien Test-Protokolle/, Trommel/trommel.txt, projekdetails, .log .mqo .obj .sym), sonst die genannten Dateien; verweigert Binaries; Name C8205_Lieferung_NNN.zip (nächste freie Nummer), Vorgabeordner /mnt/user-data/outputs. Für KI-Sitzungen	CLAUDE.md („Abgabe“)
c8205_interpreter.sh, c8205_interpreter.bat	Wrapper: starten c8205_interp.py für eine Testdatei und schreiben das Protokoll nach Test-Protokolle/<Testname>.log	C8205-Interpreter-Python_Interpreter.md, Abschnitt 5.2
testscripte/	Ordner für die Shell-Skripte, die Tests ausführen: c8205_bis_test.sh, compute_all_tests.sh und pj2-bis_minimalprogramm_test.sh sowie die Python-Hilfsskripte erzeuge_leere_trommel.py und c8205_befehlssweep.py (die folgenden Zeilen). Jedes Skript wechselt zuerst in das Projektverzeichnis (eine Ebene über testscripte/) und arbeitet dort; es darf daher aus jedem Verzeichnis gestartet werden. Pfade von Testdateien (z. B. Tests/pj2-bis_v6_druck_befehlsdruck.test) gelten relativ zum Projektverzeichnis. Die Wrapper c8205_interpreter.sh/.bat liegen weiterhin in der Wurzel. Die GRAFIK-Prüfungen in diesem Ordner beschreibt die Zeile testscripte/ (GRAFIK-Prüfungen); vollständige Liste mit je einem Satz: Abschnitt 1a	C8205-Interpreter-Durchführung_eines_Tests.md, Abschnitt 2f
testscripte/erzeuge_leere_trommel.py	Hilfsskript: erzeugt die Trommeldatei Trommel/trommel.txt mit 4096 Zeilen, alle Zellen 0 (Schalter --aus PFAD). Setzt damit auch eine benutzte Trommel zurück; der Interpreter selbst setzt sie nie zurück	C8205-Interpreter-Python_Interpreter.md, Abschnitt 8.5

C 8205 – Projektstruktur

Quelle: C8205-Projektstruktur.md · im Wortlaut der Quelle

Pfad	Zweck	Maßgebliches Dokument
`testscripte/c8205_bis_test.sh`	Wrapper für Tests der BIS-Bausteine: führt zuerst `Tests/pj2-bis_minimalprogramm_v6.test` mit `--LOADONLY` aus (Minimalprogramm v6 nur auf die Trommel laden, ohne es zu starten), dann die angegebene Testdatei; Aufruf `testscripte/c8205_bis_test.sh TESTDATEI [Schalter ...]` (vom Projektverzeichnis aus)	`C8205-Interpreter-Durchführung_eines_Tests.md`, Abschnitt 2f
`testscripte/compute_all_tests.sh`	führt den Interpreter für alle Testdefinitionen in `Tests/` aus; Soll-Dateien der Dateiprüfungen (Namen auf `-ausgabe.txt` oder `_erwartet.txt`) und `st-005` (läuft gesondert mit seiner Vorbedingung) überspringt es; die Tests des Minimalprogramms v6 (`Tests/pj2-bis_v6_*.test`, je mit eigenem Abbild) startet es in einer eigenen Schleife	`C8205-Interpreter-Durchführung_eines_Tests.md`, Abschnitt 3b.1
`testscripte/pj2-bis_minimalprogramm_test.sh`	Baut das Minimalprogramm v6 und seine Prüfprogramme (`Assembler/Projekt_2/…_test_*.asm`), lädt sie auf die Trommel und startet den GRAFIK-Interpreter; Aufruf `testscripte/pj2-bis_minimalprogramm_test.sh [--NO_GRAFIK] [BASISNAME [AUSWAHL]]` (`--NO_GRAFIK` für Regressionstests ohne Fenster)	Kopfkommentar des Skripts
`testscripte/c8205_befehlssweep.py`	Vollständigkeitstest über alle 512 Operationscodes (000 bis 777 oktal): prüft über die Knotengraph-Engine von `c8205_interp.py`, ob jeder Code eine Umsetzung hat und der Knotenautomat terminiert; prüft nicht die numerische Richtigkeit. Liegt seit der Verlegung aus `Hilfsprogramme/` in `testscripte/` (siehe Kopfkommentar zum Auffinden des Interpreters)	Kopfkommentar der Datei
`CLAUDE.md`	Karte für KI-Sitzungen (kurz, Richtwert höchstens etwa 100 Zeilen): Regeln des Projektinhabers, Schnellstart, Skriptliste, Fallen, Tabelle „Aufgabe, was lesen“. Wird zuerst gelesen. Ausnahme von der Regel, dass .md-Dateien in `markdown-dateien/` liegen	`C8205-Programmierhandbuch_fuer_Claude.md`
`C8205-Programmierhandbuch_fuer_Claude.md`	Nachschlagewerk zur Karte: Maschinenmodell, Befehlscodes, Programmieridiome, Assembler-Stolperfallen, Tests schreiben, Suchkarte, benötigte Teile des Zips. Die Anhänge A (Trommelzeit, GRAFIK) und B (BIS-Minimalprogramm) liegen in `markdown-dateien/`	`CLAUDE.md`
`README.md`, `README.en.md`	Einstiegsseiten für GitHub (deutsch, englisch); keine eigenen Regeln, sie verweisen auf die Dokumente. Ausnahme von der Regel, dass .md-Dateien in `markdown-dateien/` liegen	`C8205-Interpreter-Projektueberblick.md`

C 8205 – Projektstruktur

Quelle: C8205-Projektstruktur.md · im Wortlaut der Quelle

Pfad	Zweck	Maßgebliches Dokument
`.build-nummer`	Zähldatei im Projektstamm; kein Skript des Projekts liest oder schreibt sie (die Build-Nummer des Zips steht in `C8205-Versionierung.md`, „Letzter Build“). Nicht Teil der Dokumentation, nicht in Lieferungen aufnehmen	–
`LICENSE`	MIT-Lizenz für das Gesamtprojekt (Originalunterlagen des C 8205 ausgenommen)	–
`markdown-dateien/`	Unterordner mit den .md-Dateien des Projekts (Projektbeschreibung und Regelwerk, Quelle für Interpreter und PDFs): alle `C8205-Interpreter-*.md`-Dateien außer `C8205-Interpreter-BIS.md` und `C8205-Interpreter-Objektdatei_Listing.md` (beide im BIS-Ordner; darunter `C8205-Interpreter-PDF.md`, die Gestaltungsrichtlinie für alle PDF-Ausgaben, und `C8205-Interpreter-Lochstreifen-Hilfsprogramme.md`, die Dokumentation der beiden Lochstreifen-Hilfsprogramme), `baz-wirkungstabelle.md`, das Orientierungsdokument für neue Leser `C8205-Einstieg_fuer_Neueinsteiger.md` (gilt für das Gesamtprojekt, nicht nur für den Interpreter) sowie die beiden projektweiten Wurzeldokumente `C8205-Projektstruktur.md` (dieses Dokument) und `C8205-Versionierung.md` (Versionsnummern der Dateien). Die Rolle jeder Datei steht in der Dokumentenlandkarte von `C8205-Interpreter-Projektueberblick.md` (Abschnitt 6). Nicht hier liegen die Assemblerdokumentation (`Assembler/Dokumentation/`, s. Abschnitt 3) und die BIS-Dokumentation einschließlich des Objektdatei-Listings (`Bausteininterpretiersystem BIS/`, s. Abschnitt 3a). Jede .md-Datei existiert nur einmal. Im Unterordner `abbildungen/` liegen die Abbildungen der Dokumente (PNG, die selbst gezeichneten auch als SVG; eingebunden mit `![Bildunterschrift](abbildungen/Datei.png)`; Zeichenskript `zeichne_abbildungen.py` in `PDF/pdf-layout-scripte/`). Der Interpreter liest den Ordner über den Schalter `--mdverz` (Vorgabe: `markdown-dateien/` neben `c8205_interp.py`). Dazu `Handbuch-Tests_Testhandbuch.zip` (die Beispieltests des Testhandbuchs; eine Kopie liegt neben `build_testhandbuch.py`) sowie die Anhänge zum Programmierhandbuch `C8205-Programmierhandbuch-Anhang_A_Trommelzeit_GRAFIK.md` und `C8205-Programmierhandbuch-Anhang_B_BIS_Anschluss.md`.	Wurzel: `C8205-Interpreter-Überblick_und_Gesamtaufbau.md`; PDF-Richtlinie: `C8205-Interpreter-PDF.md` (selbst); Versionsnummern: `C8205-Versionierung.md`

C 8205 – Projektstruktur

Quelle: C8205-Projektstruktur.md · im Wortlaut der Quelle

Pfad	Zweck	Maßgebliches Dokument
`PDF/`	Erzeugte PDFs (A4), je Dokument eine Datei `C8205-<Inhalt>_A4.pdf`; hier nicht einzeln aufgezählt. Dazu gehört `C8205-Zusatztrommeln_A4.pdf` (Quelle `C8205Z/zt/Zusatztrommel_Zeitmodell.md`, Skript `erzeuge_zusatztrommel_pdf.py`). Das Assembler-PDF liegt dagegen in `Assembler/Dokumentation/` (s. Abschnitt 3), das BIS-PDF in `Bausteininterpretiersystem BIS/markdown-dateien/` (s. Abschnitt 3a) und das PDF der Testübersicht in `Testausfuehrungen/`. Die Skripte zum Erzeugen liegen im Unterordner `PDF/pdf-layout-scripte/`: je PDF ein Skript `erzeuge_<Inhalt>_pdf.py` (z. B. `erzeuge_knoten_pdfs.py` für `C8205-Knotengraph_A4.pdf` und `C8205-Knotentabellen_A4.pdf` aus den beiden Knoten-.md-Dateien, `erzeuge_rechnerbeschreibung_pdf.py` mit dem redaktionellen Text im Skript, `erzeuge_projektstruktur_pdf.py` für `C8205-Projektstruktur_A4.pdf`), `build_testhandbuch.py` für das Testhandbuch, der Markdown-Konverter `md2pdf.py` und das Zeichenskript `zeichne_abbildungen.py` für die selbst gezeichneten Abbildungen (Lampenfeld, Tastaturen; Ausgabe nach `markdown-dateien/abbildungen/`). Ein Skript ist jeweils beim Ordner seiner Quelle abgelegt: `Assembler/Dokumentation/pdf-layout-scripte/`, `Bausteininterpretiersystem BIS/markdown-dateien/pdf-layout-scripte/` und `Testausfuehrungen/pdf-layout-scripte/`; sie verwenden `md2pdf.py` aus `PDF/pdf-layout-scripte/` (oder eine Kopie daneben). Den Baustein `pdflib` (Schriften, Kopf-/Fußzeile, Deckblatt, Tabellen) gibt es nicht als eigene Datei: Jedes Skript und `md2pdf.py` tragen ihn als Quelltext eingebettet in sich; `erzeuge_testuebersicht_pdf.py` braucht weder `md2pdf.py` noch `pdflib`.	Liste, Quellen und Namensschema: `C8205-Interpreter-PDF.md`, Abschnitt 8; Skript (Aufruf, Voraussetzungen): Abschnitt 7
`make_pdf.sh`, `make_all_pdf.sh`	Aufrufhilfen für die PDF-Erzeugung (in der Wurzel): `make_pdf.sh SKRIPT [Schalter ...]` sucht das Skript `SKRIPT` in einem der Ordner `pdf-layout-scripte` (`PDF/`, `Assembler/Dokumentation/`, `Bausteininterpretiersystem BIS/markdown-dateien/`, `Testausfuehrungen/`) und startet es mit den übrigen Schaltern (z. B. `--stand 12.10.2026`); `make_all_pdf.sh` erzeugt über `make_pdf.sh` nacheinander die PDFs einer festen Skriptliste	Kopfkommentar von `make_pdf.sh`

C 8205 – Projektstruktur

Quelle: C8205-Projektstruktur.md · im Wortlaut der Quelle

Pfad	Zweck	Maßgebliches Dokument
`Bausteininterpretiersystem BIS/`	Teilprojekt BIS: Unterordner `markdown-dateien/` mit `C8205-Interpreter-BIS.md` (Dokumentation des BIS), `C8205-Interpreter-Objektdatei_Listing.md` (Objektdatei-Ausdrucke der Beispielprogramme, Vorgabebilder) und `C8205-BIS_A4.pdf` (PDF dazu) und dem Unterordner `pdf-layout-scripte/` mit `erzeuge_bis_pdf.py` (erzeugt das PDF)	Abschnitt 3a
`Tests/`	Testaufträge für den Interpreter (meist `.txt`; die Endung ist frei, der Assembler erzeugt `.test`). Unterordner `asm_erzeugt/` nimmt die von `testwahl.sh` frisch erzeugten `.test` zum Vergleichen auf; sie ersetzen nicht die Tests im Ordner selbst	`C8205-Interpreter-Durchführung_eines_Tests.md`
`Test-Ergebnisvergleiche/`	Soll-Dateien der Dateiprüfungen (Namen auf `-ausgabe.txt` oder `_erwartet.txt`), mit denen Tests ihre Ausgabe vergleichen; sie liegen nicht in `Tests/`	`C8205-Interpreter-Durchführung_eines_Tests.md`
`Test-Protokolle/`	Protokolle (`.log`) der Interpreterläufe; im Gesamt-Zip nur als `Test-Protokolle.zip` im Projektstamm (der Interpreter legt den Ordner selbst neu an)	`C8205-Interpreter-Durchführung_eines_Tests.md`, Abschnitt 8
`Test-virtuelleGeraete/`	Ablage der virtuellen Geräte für den **TEST-Interpreter** (`c8205_interp.py --MODUS=TEST`), je Gerät ein Unterordner: `Leser1/`, `Leser1_02/`, `Leser2/` (Lochstreifen für Leser 1 und 2), `Stanzer1/` (Ausgabe des Stanzers 1), `SW-Ausgabe/` (Schreibwerk-Ausgabe), `T-SW-Eingabe/` (Eingabe über Tastatur/Schreibwerk, Kanal K0); siehe Abschnitt 2	Mechanismus: `C8205-Interpreter-Durchführung_eines_Tests.md`, 6.2 und 6.3.2; Schalter: `C8205-Interpreter-Python_Interpreter.md`, 8.2 und 8.3; Trommel: dort 8.5 und `C8205-Interpreter-Durchführung_eines_Tests.md`, 2c
`virtuelle_Geraete/`	Ablage der virtuellen Geräte für den **GRAFIK-Interpreter** (`c8205_grafik.py`), Standardordner der Schalter `--leser1`, `--leser2`, `--stanzer1`: `Leser1/`, `Leser2/`, `Stanzer1/` (jeweils mit Ordner `LOCHSTREIFENKORB` bzw. `stanzer.ls`; Inhalt vom Anwender abgelegt, z. B. `.obj`/`.kep` des Assemblers)	`C8205-Interpreter-GRAFIK_Beschreibung.md`
`Trommel/`	die **gemeinsame Trommel** für TEST und GRAFIK: Trommeldatei `trommel.txt`, Schalterdatei der Bahnsperren `trommel.basp`, `trommel_r300_eingabe.txt` (der persistente Speicher des Rechners) und `Trommelvorlagen/` (Vorlagen zum Zurückkopieren: `trommel.txt`/`trommel.basp` leer, `MP.txt`/`MP.basp` mit geladenem Minimalprogramm); siehe Abschnitt 2	Trommel: `C8205-Interpreter-Python_Interpreter.md`, 8.5, und `C8205-Interpreter-Durchführung_eines_Tests.md`, 2c

C 8205 – Projektstruktur

Quelle: C8205-Projektstruktur.md · im Wortlaut der Quelle

Pfad	Zweck	Maßgebliches Dokument
`Testausfuehrungen/`	Test ausführen und Tests überblicken (der Ordner hieß früher `CSV/`). **Skripte zum Ausführen:** `testwahl.sh` listet alle Tests in `Tests/` mit Kurzinhalt, lässt einen über seine Nummer wählen und führt ihn aus (Assembler, Interpreter, Protokolldatei im `vi`; `<nnn>a` öffnet den Assembler-Quelltext); `kep_test.sh`, `interpreter_test.sh` und `bis_test.sh` sind Vorführläufe: sie leeren die Trommel, assemblieren die jeweiligen Programme (KEP, Minimalprogramm v6, Druckprogramm) und laden sie auf die Trommel bzw. führen den Test aus, `kep_test.sh` und `bis_test.sh` starten zuletzt das GRAFIK-Fenster (Bedienhinweise im Kommentar am Dateiende von `kep_test.sh` und `bis_test.sh`; welche Lademethode welchen Angang am Assemblerquelltext verlangt, steht in `C8205-Interpreter-Durchführung_eines_Tests.md`, Abschnitt 2k). Alle vier wechseln selbst in den Projektstamm (eine Ebene darüber) und dürfen von überall gestartet werden. **Übersichtstabelle aller Tests:** `test_tabelle.py` liest `Tests/` und `Assembler/` und schreibt `test_tabelle.csv`, `.md` und `.xlsx` (Schalter `--format` auch `html`; `.xlsx` nur, wenn `openpyxl` installiert ist, sonst wird es übersprungen) mit den Spalten Testdatei, Inhalt des Tests (aus den `#DOK`-Zeilen), Dateien der virtuellen Geräte bzw. Vergleichsdatei, zugehöriger ASM-Quelltext und ASM-Zuordnung (leer = Quelltext gleichen Namens, sonst die Art der Zuordnung). Das Skript findet die Basis (den Ordner mit `Tests/` und `Assembler/`) selbst, auch wenn es aus `Testausfuehrungen/` gestartet wird; die Ausgabedateien liegen im Ordner des Skripts. `test_tabelle_csv_anzeigen.sh` öffnet `test_tabelle.csv` in LibreOffice Calc und richtet die Ansicht ein (Zeilenumbruch, feste Spaltenbreiten, fette eingefrorene Kopfzeile); gelb markiert er Tests, deren Inhalt nur ein Platzhalter ist (`#DOK`-Zeile fehlt), orange Tests, deren ASM-Quelltext nur vermutet ist. Außerdem liegen hier das PDF `C8205-Testuebersicht_A4.pdf` (die Übersicht als PDF) und im Unterordner `pdf-layout-scripte/` das Skript `erzeuge_testuebersicht_pdf.py`, das es erzeugt	Kopfkommentare der Skripte; Aufruf: `bash Testausfuehrungen/testwahl.sh [-n] [Suchtext]`, `python3 test_tabelle.py [--basis PFAD] [--format csv md xlsx html]`

C 8205 – Projektstruktur

Quelle: C8205-Projektstruktur.md · im Wortlaut der Quelle

Pfad	Zweck	Maßgebliches Dokument
`Assembler/`	Teilprojekt Assembler	Abschnitt 3; Doku: `Assembler/Dokumentation/C8205_Assembler_Dokumentation.odt` (maßgeblich), `.md` (durchsuchbare Ableitung) und `C8205-Assemblerdokumentation_A4.pdf` (aus der `.md` erzeugt), gleicher Pfad
`.github/workflows/`	Abläufe für GitHub Actions: `assembler-fuzz.yml` (nächtlicher Zufallstest `make fuzz` für `Assembler/`) und `assembler.yml` (die in der Assemblerdokumentation, 6.12, genannte Prüfung bei Änderungen)	`Assembler/Regressionstest/LIESMICH.md`

1a. Die Skripte in `testscripte/`

Vollständige Liste, je Skript ein Satz. Alle Skripte wechseln selbst ins Projektverzeichnis (bzw. erwarten den Aufruf von dort, s. Kopfkommentar). Einzelheiten stehen im Kopfkommentar des Skripts und im jeweils genannten Dokument.

Tests der TEST-Seite und des BIS (`C8205-Interpreter-Durchführung_eines_Tests.md`, Abschnitte 2f und 3b.1)

Skript	Zweck
`compute_all_tests.sh`	Führt den Interpreter für alle Testdefinitionen in `Tests/` aus und meldet nur Abweichungen.
`c8205_bis_test.sh`	Lädt das BIS-Minimalprogramm v6 auf die Trommel (`--LOADONLY`) und führt dann die angegebene Testdatei aus.
`erzeuge_leere_trommel.py`	Erzeugt `Trommel/trommel.txt` mit 4096 Nullzeilen und setzt damit auch eine benutzte Trommel zurück.
`pj2-bis_minimalprogramm_test.sh`	Baut das Minimalprogramm v6 samt Prüfprogrammen, lädt sie auf die Trommel und startet GRAFIK (`--NO_GRAFIK` ohne Fenster).
`c8205_befehlssweep.py`	Vollständigkeitstest über alle 512 Operationscodes (Umsetzung vorhanden, Automat terminiert).

Prüfungen des Modus GRAFIK im Sammellauf (`C8205-Interpreter-GRAFIK_Pruefverfahren.md`, §3)

Skript	Zweck
`grafik_alle_tests.sh`	Startet alle Prüfblöcke nacheinander (`--schnell` ohne die Gleichwertigkeitsläufe) und meldet je Block OK oder FEHLER; der letzte Block meldet FEHLER, wenn sich `Trommel/trommel.txt` oder `trommel.basp` während des Laufs geändert hat (OP-19).
`grafik_kern_tests.py`	Prüft den GRAFIK-Kern: Dauerbetrieb am Pult, Unterbrechung durch H, K0-Warteschlange mit zwei Quellen, Abbruch, Bahnsperre und Schalterdatei.
`grafik_geraete_tests.py`	Prüft Leser (Ordner, Änderungszeit, `LOCHSTREIFENKORB`, aktives Warten) und Stanzer (`stanzer.ls` fortschreiben) und ihr Zusammenspiel mit dem Rechner.
`grafik_checkpoint4_tests.py`	Prüft, dass Gerätefehler den Rechner nicht abbrechen, H in Endlosschleifen wirkt und der Stanzer fortschreibt statt zu überschreiben.
`grafik_checkpoint5_tests.py`	Prüft Fehlerfälle beim Verschieben in den `LOCHSTREIFENKORB` und weitere Randfälle der Leser.
`grafik_gleichwertigkeit.py`	Spielt Tests aus `Tests/` im TEST-Verfahren und im GRAFIK-Kern ab und vergleicht die Ergebnisse (`--neue-geraete` mit den neuen Geräteklassen).
`grafik_pult_tests.py`	Prüft die Pult-Steuerung ohne Tkinter und die Tk-Schicht gegen ein Ersatzmodul.
`grafik_trommel_waechter.py`	Wächter zu OP-19: merkt die Prüfsummen von `Trommel/trommel.txt` und `trommel.basp` (`merken`) und vergleicht sie (`pruefen`); vom Sammellauf aufgerufen, ändert nichts.
`grafik_trommel_waechter_tests.py`	Prüfungen des Wächters mit Gegenproben (Temp-Ordner); Block des Sammellaufs.

C 8205 – Projektstruktur

Quelle: C8205-Projektstruktur.md · im Wortlaut der Quelle

Skript	Zweck
op09_eingabeklang.sh	Handprüfung H-7 (OP-09): lädt den Abschnitt Trommeldatei: eines Tests (Vorgabe sw_zwei_ziffern) auf eine frische leere Trommelkopie op09_trommel.txt und startet GRAFIK mit Fenster, Ton und --DEBUGLOG op09_debug.log; --nur-vorbereiten ohne Fenster. Trommel/trommel.txt bleibt unberührt.
grafik_e_im_eingabe_halt_probe.py	Probe zu OP-20 (behoben 21.10.2026): stellt H im Eingabe-Halt eines E-Schritts nach und gibt den Zustand aus (E wirkt wieder, K0 wartet nicht mehr); ändert nichts, nicht Teil des Sammellaufs. Die Prüfungen stehen in grafik_pult_tests.py.
grafik_op09_rechnerzeit_probe.py	Probe zu OP-09: misst die Zeit des Rechners vom Umschaltzeichen bis zum nächsten 011 mit Schreibwerkzeit (Test sw_zwei_ziffern auf Trommelkopie im Temp-Ordner); Ausgabe, kein Block des Sammellaufs.
grafik_sw_zeit_messung.py	Rechnet aus der Schreibwerk-Ausgabe der Tests aus Tests/ die Schreibwerkzeit aus, die ein Lauf über den echten Treiber abwarten würde (OP-40); ändert nichts, nicht Teil des Sammellaufs.
grafik_sektor_pruefung.py	Prüft, ob die Befehlsschleife des GRAFIK-Interpreters schnell genug ist, damit zeitoptimal programmierter Code seinen Sektor findet (Sprungketten auf 0000 bis 0037, Befehlsart sprung, einzel oder gruppe); ändert nichts, nicht Teil des Sammellaufs.
grafik_trommel_zaehler_probe.py	Probelauf zu den Zeitmodellen (alt, echt, schnell) und den Zählern der Trommelstatistik (GRAFIK_Beschreibung §7.8) auf einer Kopie der Trommel im Temp-Ordner; ändert das Projekt nicht, kein Block des Sammellaufs.
grafik_pult_stress.py	Klickt zufällig auf alle Pult-Tasten gegen den echten Rechner-Thread und meldet Hänger über 3 s.
grafik_sw_tests.py	Prüft das SW-Fenster: Logik ohne Tkinter, Anbindung an die K0-Warteschlange, Tk-Schicht mit Ersatzmodul.
grafik_sw_pult_test.py	Prüft die SW-Eingabe über die Pult-Tasten (Netz, SW, G, R) mit dem Programm sw_zwei_ziffern.
grafik_sw_ende_zu_ende.py	Lässt ein Ziffernprogramm auf dem echten GRAFIK-Rechner mit der Logikschicht des SW-Fensters laufen und vergleicht Ausgabe und Register.
grafik_sw_abstand_tests.py	Prüft den Abstand zwischen Umschalttaste und Zeichentaste bei gehaltener Umschalttaste.
grafik_tastatur_tests.py	Prüft den Schalter --TASTATUR=de|en ohne Tkinter.
grafik_stufe4_tests.py	Prüft die Töne: Mischer, Schreibwerkzeit, Klangwahl, Selektor SW, Ausfall (Code 16), ohne Audiogerät.
grafik_stufe5_tests.py	Prüft die Umschaltung --MODUS in c8205_interp.py (Standard GRAFIK, TEST wie bisher).

Prüf- und Messwerkzeuge außerhalb des Sammellaufs (C8205-Interpreter-GRAFIK_Pruefverfahren.md, §3.3 und §5)

Skript	Zweck
grafik_sw_tk_test.py	Testet das echte SW-Fenster und das HW-Check-Fenster in einem Prozess; im Sammellauf läuft nur die Variante --ohne-tk.
grafik_pult_tk_test.py	Testet die Pult-Tastenbedienung am echten Tk-Fenster (Verfahren in §5 des Prüfverfahrens).
grafik_sw_vormerkung_s3_13.py	Prüfmittel zu S3-13/S3-18: keine Vormerkung, Sperre statt Warten, Eingabe nur bei aktivem 011.
grafik_zeit_messung.py	Misst, wie stark Pult-Fenster, Lampenabtastung und HW-Check-Fenster die Echtzeit der Befehlsausführung stören (Prüfverfahren §7.7).
k0_schatten_haken.py	Messwerkzeug: baut in eine Kopie von c8205_interp.py einen Zusatz ein, der Abweichungen zwischen alter und neuer K0-Regel protokolliert (nie auf das Projekt selbst anwenden).

Eingabeprogramm `r300_eingabe` (`C8205-Interpreter-GRAFIK_Pruefverfahren.md`, §6)

Skript	Zweck
`r300_eingabe_erzeugen.py`	Erzeugt aus der Zeichentabelle Quelltext, Testauftrag, K0-Datei, Klickspur und Sollfolge des Eingabeprogramms.
`r300_eingabe_trockenlauf.py`	Prüft den GRAFIK-Weg des Eingabeprogramms ohne Tkinter gegen die Sollfolge (Rückgabewert 0 = in Ordnung).
`r300_eingabe_grafik.sh`, `r300_eingabe_grafik.bat`	Starten GRAFIK mit `--HWCHECK` und `--WIEDERGABE` für den echten Lauf und schreiben das Debug-Log.
`r300_eingabe_lauf_und_auswertung.sh`, `r300_eingabe_lauf_und_auswertung.bat`	Führen den echten Lauf und die Auswertung in einem Schritt aus.
`r300_eingabe_log_auswerten.py`	Wertet das Debug-Log des Laufs gegen Klickspur und Sollfolge aus (Rückgabewert 1 bei mindestens einem Befund).

2. Konventionen für Tests, Protokolle und virtuelle Geräte

- **Testname:** `NNNN-<Operationscodes>.txt` in `Tests/` (z. B. `0004-210_544_020.txt`). Testaufträge aus Assemblerläufen heißen `asm_NNNN_<Programmname>.txt` (z. B. `asm_0001_bsp_5_16.txt`). Weitere Namensfamilien in `Tests/`: `bsp_…`, `sp-…`, `tk-…`, `st-…`, `int-…`, `ziffern_…`, `lb-…`, `ac-…`, `bz-…`, `acbz-…`, `wr-…`, `trommel-…` und `s1-…` bis `s4-…`, `bs-…`, `sw_…`, `r300_…`, `namen_…`, `k1_…`, `betrag…`, `bdpt1…`, `knotenlimit_…`, `v6_…` (Bausteintests des Minimalprogramms ohne Präfix `pj2-bis_`), `asm_druck_…`, `test_vorspann…` (Vorspann der Testaufträge), `pj2_…` (Programmbeispiele mit Unterstrich: ggT, Maximum, Vertauschung) und Einzeltests ohne Präfix aus Assemblerläufen (`bedsprung`, `code`, `code_wdhl_bef`, `dt_test`, `marke_allein`, `mini`, `sortierung`, `deepseek_asm_20261007_…`) sowie die BIS-Tests `pj2-bis_…` (s. u.); ihr Inhalt steht jeweils in den `#DOK`-Zeilen der Testdatei (Übersicht: `Testausfuehrungen/`). Das Protokoll trägt denselben Namen mit der Endung `.log` in `Test-Protokolle/`. Der Wrapper findet die Testdatei über den Namen.
- **Virtuelle Geräte:** Der Interpreter kennt die Geräte nur über die Schalter `--leser1`, `--leser2`, `--stanzer1`, `--sw-ausgabe` und `--k0`. Diese nehmen einen Pfad (Datei oder Ordner) entgegen. Die Verzeichnisse `Test-virtuelleGeraete/Leser1/`, `…/Leser2/`, `…/Stanzer1/`, `…/SW-Ausgabe/` und `…/T-SW-Eingabe/` sind die dafür vorgesehene Ablage; **die Wrapper übergeben sie nicht**; die Testdatei darf die Zuordnung aber selbst als Zeile enthalten (`C8205-Interpreter-Durchführung_eines_Tests.md`, Abschnitt 2b). Sie wirken nur, wenn der Schalter beim Aufruf angegeben wird, z. B. `--leser1 Test-virtuelleGeraete/Leser1` oder `--k0 datei=Test-virtuelleGeraete/T-SW-Eingabe`. Für den Interpreter GRAFIK gelten die Standardordner `virtuelle_Geraete/Leser1/`, `…/Leser2/` und `…/Stanzer1/` (eigene, anfangs leere Ordner); die Trommel `Trommel/trommel.txt` benutzen beide Interpreter gemeinsam.
- **BIS-Tests:** Tests, die einen BIS-Baustein prüfen, tragen das Präfix `pj2-bis_` (z. B. `pj2-bis_v6_druck_befehlsdruck.test`, Soll-Datei `pj2-bis_v6_druck_befehlsdruck_erwartet.txt`). Die Tests des Minimalprogramms v6 heißen `pj2-bis_v6_*.test` und tragen ihr Abbild selbst (`#TROMMEL=LEER`); Tests künftiger Bausteine setzen das Minimalprogramm auf der Trommel voraus, das `pj2-bis_minimalprogramm_v6.test` zuerst laden muss (`c8205_bis_test.sh`, mit `--LOADONLY`); Einzelheiten: `C8205-Interpreter-Durchführung_eines_Tests.md`, Abschnitt 2f.
- **Trommel:** Die Trommel ist die Datei `Trommel/trommel.txt` (4096 Zeilen). Anders als die übrigen Geräte ist sie nicht optional: Der Interpreter braucht sie bei jedem Aufruf. Vorgabe ist der Pfad neben `c8205_interp.py`; ein anderer Pfad wird mit `--trommel` angegeben (die Wrapper übergeben den Schalter nicht, es gilt die Vorgabe). Die Datei wird mit `testscripte/erzeuge_leere_trommel.py` erzeugt. Sie **behält ihren Inhalt** über Tests und Aufrufe hinweg: Die Zeilen einer Testdatei überlagern sie, nicht genannte Zellen behalten ihren Wert, jede Zelländerung eines Befehls wird sofort in der Datei nachgeführt. Der Interpreter setzt sie nie zurück. Tests, die Zellen prüfen oder lesen, die sie nicht selbst belegen, brauchen daher eine leere Trommel (`C8205-Interpreter-Durchführung_eines_Tests.md`, Abschnitt 2c).

3. Teilprojekt Assembler

Der Assembler `c82ass` übersetzt Assembler-Quelltext (`.asm`) in Objektcode für das "Kleine Eingabeprogramm". Aufruf, Sprache, Makros und Beispiele stehen in der Assemblerdokumentation; hier nur die Struktur.

Pfad (unter `Assembler/`)	Zweck
`Makefile`	Erzeugt und prüft den Assembler (`make`, `make test`, `make pruefen`, `make fuzz`, `make clean`); die Liste der Quelldateien steht nur hier

C 8205 – Projektstruktur

Quelle: C8205-Projektstruktur.md · im Wortlaut der Quelle

Pfad (unter `Assembler/`)	Zweck
`make_c82ass.sh`	Skript zum Erzeugen des Assemblers unter Linux ohne `make`
`make_c82ass.bat`	Windows-Pendant zu `make_c82ass.sh` (erzeugt `_Windows/c82ass.exe`; kein Wrapper zum Ausführen)
`c82ass.sh`	Ausführen des Assemblers; aus dem Ordner `Assembler/` zu starten (ruft `./_Linux/c82ass` relativ zum aktuellen Verzeichnis auf)
`asm_alle.sh`	assembliert alle `.asm`-Dateien unterhalb eines Verzeichnisses (Vorgabe: `Assembler/`) mit `c82ass.sh`; meldet nur Fehler. Wechselt zuerst selbst in den Ordner `Assembler/`, darf also von überall gestartet werden
`asm_und_test.sh`	assembliert ein Programm, ergänzt die erzeugte `.test` (`#TROMMEL=LEER`, `#SPEI`, `Prüfungen:` mit `US=1` und den genannten Ergebniszellen `AAAA=WERT`) und startet den Interpreter; darf von überall gestartet werden; Rückgabecode wie der Interpreter, bei Assemblerfehler 16
`asm_aufraeumen.sh`	löscht in den Projektordnern die vom Interpreter nicht benutzten Dateien `.log`, `.mqo`, `.obj`, `.sym` (Trockenlauf ohne Schalter, `--loeschen` löscht)
`Dokumentation/`	Assemblerdokumentation `C8205_Assembler_Dokumentation`: `.odt` (maßgeblich), `.md` (durchsuchbare Ableitung daraus, kein eigenes Regelwerk), `C8205-Assemblerdokumentation_A4.pdf` (aus der `.md` erzeugt, nach der PDF-Richtlinie); im Unterordner `pdf-layout-scripte/` liegt das Erzeugungsskript `erzeuge_assemblerdokumentation_pdf.py`
`Sourcecode/`	C-Quellcode des Assemblers selbst, in Module zerlegt (`ca82_main.c` Ablauf und Optionen, `ca82_quelle.c` Einlesen, `ca82_makros.c` Makros, `ca82_pass0.c` bis `ca82_pass3.c` die Pässe, `ca82_ausdruck.c`, `ca82_codierung.c`, `ca82_symbole.c`, `ca82_listing.c`, `ca82_testdatei.c`, `ca82_berichte.c` für `-WARN=` und `--xref`, `ca82_fluss.c` für `-WARN=SPR/TOT`, dazu Fehler-, Hilfs- und Protokollmodule und `c82_austausch.c`); die Versionsnummer steht nur in `ca82_version.h`. Enthält außerdem die Makrodatei `c82ass.makros` (der Assembler sucht sie unter diesem festen Namen zuerst im Arbeitsverzeichnis des Aufrufs, sonst in `Sourcecode/`; hier liegt sie richtig)
`Regressionstest/`	Automatische Prüfung des Assemblers: `regressionstest.sh` (alle `.asm` gegen die Referenz in `referenz/`, dazu die Negativtests in `negativ/`), `pruefe_alles.sh` (mit `-Werror` und Sanitizern, auch für CI), `fuzz.py` (Zufallstest); Beschreibung in `LIESMICH.md`. Die Skripte schreiben nichts ins Projekt. Im Gesamt-Zip fehlt der Ordner; er liegt als `Regressionstest.zip` im Projektstamm (Pfade darin: `Assembler/Regressionstest/...`) und wird dort entpackt, bevor man ihn braucht. `c8205_zip_packen.sh` erzeugt es
`_Linux/`, `_Windows/`	Ziele für das Binary des Assemblers je Betriebssystem: `_Linux/` enthält `c82ass`; `_Windows/` ist leer und wird von `make_c82ass.bat` mit `c82ass.exe` gefüllt
`projekdetails`, `projekdetails.<Projekt>`	Arbeitsdateien: zuletzt benutzte Aufrufparameter, bzw. für ein bestimmtes Projekt gesicherte
`Projekt_1/`, `Projekt_2/`, `Proj_FeldSortieren/`, `Claude/`	Projektverzeichnisse (Beispiele); dürfen beliebig benannt sein und Unterverzeichnisse enthalten. `Claude/mini/` ist das kleinste geprüfte Beispiel (`mini.asm`, `mini.test`; Aufruf siehe `CLAUDE.md`)
`Claude_bsp_reassemblierung/`	Projektverzeichnisse, je eines pro Programm; enthält u. a. die Quelltexte der Tests aus `Tests/`

Vom Assembler erzeugte Dateien (im Verzeichnis der `.asm`-Datei, Namensstamm wie die Quelle; der Assembler löscht `.log`, `.obj`, `.mqo`, `.sym`, `.test` und `.dmp` vor jedem Lauf und erzeugt sie neu):

Endung	Inhalt
`.asm`	Quelltext (Eingabe, keine erzeugte Datei)
`.log`	Literaltabelle, Symboltabelle, Tabelle der absoluten Symbole; mit `-WARN=` oder `--xref` zusätzlich deren Berichte
`.obj`	binär codierter R300-Lochstreifen mit dem Objektcode (K1K2), ohne Leerzeichen und ohne Zeilenende; unmittelbar in den Ordner des Lochbandlesers legbar
`.mqo`	aufbereiteter Quelltext, kombiniert mit dem Maschinencode (Hilfsdatei, Pass 3)
`.sym`	Symboldatei (ordnet Symbole zu Trommeladressen zu)
`.dmp`	formatierte Speicherbelegung (Listing mit Adresse, Maschinenwort, Label, Mnemonic, Operand, Kommentar); entsteht in Pass 3
`.test`	Trommeldatei im Format des Interpreters (`Adresse: Wert # Quelltextzeile`) mit Platzhalter-Definitionen für Testmarken und Tastenfolge sowie ggf. dem Leerbefehl 164, die für den jeweiligen Fall ergänzt werden müssen; Grundlage für Testaufträge, s. `C8205-Interpreter-Durchführung_eines_Tests.md`, Abschnitt 2a

3a. Teilprojekt BIS

Das Bausteininterpretationssystem BIS ist das dritte Teilprojekt (Zweck und Inhalt: `C8205-Interpreter-Projektueberblick.md`, Abschnitt 3a). Es besteht derzeit **nur aus Dokumentation**: kein Quellcode, kein Binary, keine erzeugten Dateien. Die Dokumentation liegt im Ordner `Bausteininterpretiersystem BIS/markdown-dateien/` als `C8205-Interpreter-BIS.md` (maßgeblich für das BIS) und `C8205-Interpreter-Objektdatei_Listing.md` (Format der Objektdatei, die das KEP liest, und Weg vom Vorgabebild zum Testauftrag); das zugehörige PDF `C8205-BIS_A4.pdf` liegt im selben Ordner, ebenso der Unterordner `pdf-layout-scripte/` mit dem Erzeugungsskript `erzeuge_bis_pdf.py`. Der Interpreter und der Assembler benutzen das BIS nicht. Das Minimalprogramm der BIS-Tests (`pj2-bis_minimalprogramm_v6`) liegt als Assemblerprojekt in `Assembler/Projekt_2/`, die Testaufträge in `Tests/` (Abschnitt 2, „BIS-Tests“). Werden später BIS-Bausteine abgelegt (Quelltexte, Trommeldateien, Lochband-Objektcode), ist dafür hier ein Verzeichnis nach dem Muster von `Assembler/` festzulegen und in diesem Abschnitt zu beschreiben.

3b. Experimenteller Prototyp C8205Z

`C8205Z/` enthält den **experimentellen Prototyp** der vier Zusatztrommeln des C 8205Z (Rekonstruktion; was der Inhaber festgelegt hat und was Annahme ist, steht in `C8205Z/zt/Zusatztrommel_Zeitmodell.md`). Es ist der einzige Zusatzpfad im Projektverzeichnis; der Assembler wird dafür nicht verändert. Ohne den Schalter `--C8205Z` bleibt der Simulator unverändert.

Pfad	Zweck
`C8205Z/zt/`	`Zusatztrommel_Zeitmodell.md` (Eckdaten, Ablauf, Zeiten, Befehle `5k00 030`, `zzz0 bbbb 601` Ausgabe, `zzz0 bbbb 661` Eingabe, Modulspezifikation, Referenzwerte, Festlegungen) und `zt_zeitmodell.py` (Rechnung des Zeitmodells)
`C8205Z/zt_modul/`	C-Modul der Zusatztrommeln (`zt_modul.c/.h`, Testrahmen `testframe_zt.c`, `Makefile`: `make test`) und `python/` mit dem Erweiterungsmodul `c8205_ztmodul` (`c8205_ztmodul.c`, `setup.py`; bauen: `cd C8205Z/zt_modul/python && python3 setup.py build_ext --inplace`; das Binary liegt nicht im Zip)
`C8205Z/Tests_Z/`	Tests `zt-001` bis `zt-003` und die Skripte `zt_tests.sh` (TEST), `zt_grafik_tests.sh` (TEST gegen GRAFIK in den drei Trommelzeiten) und `zt_hwcheck_test.py` (Anzeige im HW-Check, ohne Fenster); Aufruf vom Projektverzeichnis: `bash C8205Z/Tests_Z/zt_tests.sh`
`C8205Z/pult_z/`	Zeichnung des Bedienpults der Z-Ausführung (SVG, PNG, `elemente.csv`, `gen.py`, `aufgabe_pult.md`); ruht, wartet auf Fotos
`C8205Z/aufgabe`	Arbeitsstand des experimentellen Prototyps

Anbindung im Simulator: siehe die Zeilen `c8205_zt.py`, `grafik_zt.py` und `Trommel/ZT/` in Abschnitt 1. Der HW-Check (`--HWCHECK`) zeigt mit `--C8205Z` je Zusatztrommel die zuletzt bearbeitete Adresse (grüner Punkt hinter der angewählten) und ein Inhaltsfeld.

4. Offene Punkte

Keine offenen Punkte.

Erledigt:

- **Assembler-Abschnitt (0.95-013, 21.10.2026):** Zeilen für `Makefile`, `Regressionstest/`, `.github/workflows/` und die Module in `Sourcecode/` ergänzt, `.log` um die Berichte erweitert (Assembler 1.00-026).
- **Bauskripte:** `c82ass` und `c82ass.bat` heißen jetzt `make_c82ass.sh` und `make_c82ass.bat` (Abschnitt 3). Die Assemblerdokumentation nennt die neuen Namen (.odt, Abschnitte 1.1 und 2); `.md` und `.pdf` sind daraus nachgeführt.
- **Assemblerdokumentation (.odt):** Änderungen (Passbeschreibung, Dateiliste, Streichung `.mqt`/`.l82`, `.dmp`, `.test`, Makrodatei, Beispiellauf, Aussage zu `c82ass.bat`) in die `.odt` übernommen. Das `.pdf` wurde daraus neu exportiert, die `.md` daraus nachgeführt.
- **Vorhandene `.mac`-Dateien:** vom Projektinhaber entfernt.
- **Ablageort der Makrodatei:** kein Problem. `erzeugeMacroArray` sucht `c82ass.makros` zuerst im Arbeitsverzeichnis und weicht auf `Sourcecode/` aus; die Ablage in `Assembler/Sourcecode/` ist damit richtig.
- `c82ass.sh`/`make_c82ass.bat` (die `.bat` hieß vormals `c82ass.bat`): geprüft; `.sh` startet nur das Binary, `.bat` erzeugt nur `c82ass.exe`.
- **Endungen `.dmp`, `.mac`, `.test`:** `.dmp` ist die formatierte Speicherbelegung (Pass 3), `.test` die Trommeldatei für den Interpreter (Abschnitt 3, Tabelle; ebenso in der Assemblerdokumentation), `.mac` wird nicht erzeugt.
- `.mqt`, `.l82`**:** vom aktuellen Assembler nicht mehr erzeugt, in der Assemblerdokumentation gestrichen.
- `c82load`, `c82dump`, `c82simi`, `C8205Simulator`**:** Diese Programme gehören nicht zum Projekt. Die Assemblerdokumentation (Abschnitt 1) erwähnt sie nur als Abnehmer von Assemblerdateien.